



LeetCode 907: Sum of Subarray Minimums

1. Problem Title & Link

Title: LeetCode 907: Sum of Subarray Minimums

Link: <https://leetcode.com/problems/sum-of-subarray-minimums/>

2. Problem Statement (Short Summary)

Given an integer array:

arr

For every possible subarray:

- Find the minimum element.
- Add all those minimum values.

Return the sum modulo:

$10^9 + 7$

3. Examples (Input → Output)

Example 1

Input:

arr = [3,1,2,4]

Output:

17

Explanation:

Subarrays:

Subarray	Minimum
[3]	3
[3,1]	1
[3,1,2]	1
[3,1,2,4]	1
[1]	1
[1,2]	1
[1,2,4]	1
[2]	2
[2,4]	2
[4]	4

Sum:

$3+1+1+1+1+1+1+2+2+4 = 17$

**Example 2**

Input:

arr = [11,81,94,43,3]

Output:

444

Example 3

Input:

arr = [1]

Output:

1

4. Constraints

- $1 \leq \text{arr.length} \leq 3 \times 10^4$
- $1 \leq \text{arr}[i] \leq 3 \times 10^4$

Brute force:

$O(n^2)$

or

$O(n^3)$

will TLE.

5. Core Concept (Pattern / Topic)**Monotonic Stack**

Related Topics:

- Previous Smaller Element
- Next Smaller Element
- Contribution Technique
- Stack

6. Thought Process (Step-by-Step Explanation)**Brute Force**

Generate all subarrays.

For each subarray:

find minimum

Complexity:

$O(n^3)$

Even optimized:



$O(n^2)$

Still too slow.

Key Observation

Instead of asking:

For every subarray,
what is its minimum?

Ask:

For every element,
how many subarrays
use it as minimum?

This is called:

Contribution Technique

Example

arr = [3,1,2,4]

Element:

1

acts as minimum for:

[1]

[3,1]

[1,2]

[1,2,4]

[3,1,2]

[3,1,2,4]

Total:

6 subarrays

Contribution:

$1 \times 6 = 6$

Finding Contribution

For every element:



Find:

Previous Smaller Element

Distance on left.

Next Smaller Element

Distance on right.

Then:

Contribution

=

$arr[i]$

×

leftChoices

×

rightChoices

Why Multiplication?

Choose:

Any valid left boundary

and

Any valid right boundary

independently.

Hence:

left × right

subarrays.

7. Visual / Intuition Diagram (ASCII Diagram)

```

Array:
3  1  2  4
   ↑
For element:
1
Previous Smaller:
None
Distance:
2
Choices:
[1]
```



[3,1]

Next Smaller:

None

Distance:

3

Choices:

[1]

[1,2]

[1,2,4]

Contribution:

$1 \times 2 \times 3$

=

6

8. Pseudocode (Language Independent)

Find previous smaller distances

Find next smaller distances

answer = 0

for every element:

 contribution =
 value $\times$ left $\times$ right

 add contribution

return answer % MOD

9. Code Implementation

 Python

```
class Solution:

    def sumSubarrayMins(self, arr):

        MOD = 10**9 + 7
        n = len(arr)

        left = [0] * n
```

```
right = [0] * n

stack = []

for i in range(n):

    while (
        stack and
        arr[stack[-1]] > arr[i]
    ):
        stack.pop()

    if stack:
        left[i] = i - stack[-1]
    else:
        left[i] = i + 1

    stack.append(i)

stack = []

for i in range(n - 1, -1, -1):

    while (
        stack and
        arr[stack[-1]] >= arr[i]
    ):
        stack.pop()

    if stack:
        right[i] = stack[-1] - i
    else:
        right[i] = n - i

    stack.append(i)

answer = 0

for i in range(n):

    answer += (
        arr[i]
        * left[i]
        * right[i]
    )

return answer % MOD
```



```
import java.util.*;

class Solution {

    public int sumSubarrayMins(
        int[] arr
    ) {

        int n = arr.length;

        long MOD = 1_000_000_007L;

        int[] left = new int[n];
        int[] right = new int[n];

        Stack<Integer> stack =
            new Stack<>();

        for (int i = 0; i < n; i++) {

            while (
                !stack.isEmpty() &&
                arr[stack.peek()] > arr[i]
            ) {
                stack.pop();
            }

            left[i] =
                stack.isEmpty()
                ? i + 1
                : i - stack.peek();

            stack.push(i);
        }

        stack.clear();

        for (int i = n - 1;
            i >= 0;
            i--) {

            while (
                !stack.isEmpty() &&
                arr[stack.peek()] >= arr[i]
            ) {
                stack.pop();
            }

            right[i] =
                stack.isEmpty()
                ? n - i
```

```
        : stack.peek() - i;

        stack.push(i);
    }

    long answer = 0;

    for (int i = 0; i < n; i++) {

        answer +=
            (long) arr[i]
            * left[i]
            * right[i];
    }

    return (int) (answer % MOD);
}
}
```

10. Time & Space Complexity

Time Complexity

Previous Smaller:

$O(n)$

Next Smaller:

$O(n)$

Contribution:

$O(n)$

Total:

$O(n)$

Space Complexity

$O(n)$

For stack and arrays.

11. Common Mistakes / Edge Cases

Mistake 1

Using:

>

in both loops.

Duplicate handling becomes incorrect.

**Mistake 2**

Forgetting modulo.

Required:

10^9+7

Mistake 3

Using brute force.

Will TLE.

12. Detailed Dry Run (Step-by-Step Table)

Input:

arr = [3,1,2,4]

Left Distances

Index	Value	Left
0	3	1
1	1	2
2	2	1
3	4	1

Right Distances

Index	Value	Right
0	3	1
1	1	3
2	2	2
3	4	1

Contributions

Value	Left	Right	Contribution
3	1	1	3
1	2	3	6
2	1	2	4
4	1	1	4

Total:

$3+6+4+4$

=



17

13. Common Use Cases (Real-Life / Interview)

Financial Analytics

Minimum stock prices across intervals.

Resource Monitoring

Lowest utilization across time windows.

Temperature Analysis

Minimum temperature in periods.

Performance Tracking

Worst-case metrics in ranges.

14. Common Traps (Important!)

- Wrong duplicate handling
- Forgetting contribution logic
- Using $O(n^2)$ solution
- Missing modulo
- Incorrect distance calculation

15. Builds To (Related LeetCode Problems)

1. LC 496 — Next Greater Element I
2. LC 739 — Daily Temperatures
3. LC 901 — Online Stock Span
4. LC 84 — Largest Rectangle in Histogram
5. LC 907 — Sum of Subarray Minimums

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Brute Force	$O(n^3)$	$O(1)$	Impossible
Optimized Brute Force	$O(n^2)$	$O(1)$	TLE
Monotonic Stack	$O(n)$	$O(n)$	Optimal

17. Why This Solution Works (Short Intuition)



Instead of computing the minimum for every subarray, we count how many subarrays consider each element as their minimum. Using Previous Smaller and Next Smaller elements, we determine each element's contribution in $O(1)$, leading to an overall $O(n)$ solution.

18. Variations / Follow-Up Questions

Follow-Up 1

What if we need:

Sum of Subarray Maximums?

Use:

Previous Greater

Next Greater

instead.

Follow-Up 2

What if we need:

Maximum - Minimum

for every subarray?

See:

LC 2104 – Sum of Subarray Ranges

Follow-Up 3

What if updates occur dynamically?

Use:

Segment Tree

or advanced range query structures.